

Pandas 速查手册(第 1-9 节简化版)

用途:背诵 / 速查 / 面试前复习

特点:只讲是什么、干什么用,不举例不展开

配套:详细版见《Pandas完整笔记_第1-9节.md》

第 1 节:核心概念

定位

- **Pandas = 内存数据库 + 表格运算 + 数据分析瑞士军刀**
- 底层基于 NumPy
- 接口模仿 SQL/Excel/R

与 NumPy 的区别

- 有**行/列标签**(Index)
- 每列可以是**不同类型**(异构数据)
- 高级操作简洁(分组、合并、透视、时间序列)

两大数据结构

- **Series**:带标签的一维数组(类似一列 Excel)
- **DataFrame**:带行列标签的二维表(类似整个 Excel 表)

Series 核心属性

- `s.values` / `s.index` / `s.dtype` / `s.shape` / `s.size` / `s.name`

DataFrame 核心属性

- `df.values` / `df.index` / `df.columns` / `df.dtypes` / `df.shape`

Index 是灵魂

- 快速查找(用标签直接索引)
- **自动对齐**:两个 Series 相加按 index 对齐(NumPy 按位置)
- 保持元数据

创建方式 5 种

- 从 list/array
 - **从字典**(最常用)
 - 从 list of dicts
 - 从文件(read_csv 等)
 - 从 NumPy 数组
-

第 2 节:数据 IO

读文件

- `pd.read_csv` / `pd.read_excel` / `pd.read_json` / `pd.read_parquet` / `pd.read_sql`
- `pd.read_html` (网页表格)
- `pd.read_clipboard` (剪贴板,临时分析神器)

常用参数

- `sep` / `encoding` (中文文件常用 `gbk`)
- `header` / `index_col` / `usecols` / `nrows`
- `parse_dates` / `dtype` / `na_values`

写文件

- `df.to_csv('out.csv', index=False)`
- `df.to_excel` / `df.to_parquet` / `df.to_json`
- 给 Excel 看的 CSV 用 `encoding='utf-8-sig'`

探索 6 板斧

- `.shape` 形状
 - `.head()` / `.tail()` 头尾
 - `.info()` 类型 + 非空 + 内存
 - `.describe()` 数值统计摘要
 - `.dtypes` 每列类型
 - `.value_counts()` 类别频次
-

第 3 节:选择数据 ★★

选列

- `df['列']`: 单列(Series)
- `df[['列1', '列2']]`: 多列(DataFrame, 双层方括号)

选行

- `.loc[标签]`: 按标签, 切片右闭
- `.iloc[位置]`: 按位置, 切片右开

同时选行列

- `df.loc[行标签, 列标签]`
- `df.iloc[行位置, 列位置]`

布尔索引

- `df[df['年龄'] > 30]`
- 多条件用 `&` `|` `~`, 每个条件加括号
- `isin([...])`: 类似 SQL 的 IN
- `~isin([...])`: NOT IN

字符串筛选

- `df['列'].str.contains/startswith/endswith`
- `df['列'].str.match(正则)`

query() SQL 风格

- `df.query('年龄 > 30 and 工资 < 20000')`
- 引用变量用 `@变量名`

SettingWithCopyWarning 处理

- 想改副本: `df_subset = df[条件].copy()`
 - 想改原 df: `df.loc[条件, '列'] = 值`
-

第 4 节:数据清洗

缺失值

- `df.isna()` / `df.notna()`
- `df.isna().sum()` :每列缺失数
- `df.isna().mean()` :每列缺失率
- `df.dropna(subset=['col'])` / `df.dropna(thresh=3)`
- `df.fillna(0/mean/median/mode)`
- `df.ffill()` 前向填充 / `df.bfill()` 后向填充

重复值

- `df.duplicated()` :返回布尔 Series
- `df.drop_duplicates(subset=, keep='first/last/False')`

类型转换

- `.astype(int/float/str/bool)`
- `pd.to_datetime(s, errors='coerce')`

- `pd.to_numeric(s, errors='coerce')` (脏数据救星)

字符串处理(`.str` 访问器)

- `.str.lower/upper/title/capitalize`
- `.str.strip/lstrip/rstrip`
- `.str.contains/startswith/endswith/match`
- `.str.replace(旧, 新, regex=True)`
- `.str.split(' ', expand=True)`
- `.str.extract(r'(\w+)')`
- `.str.len()`

增删改列

- 增: `df['新列'] = ...`
- 删: `df.drop('列', axis=1)` / `del df['列']`
- 改名: `df.rename(columns={...})`
- 整体改: `df.columns = ['新A', '新B']`

替换值

- `df['列'].replace({'M': '男', 'F': '女'})`
 - `df.replace({'?': np.nan})`
-

第 5 节:apply / map / transform

Series.apply

- 对每个元素应用函数
- `s.apply(func)` / `s.apply(lambda x: ...)`

Series.map

- 用字典映射(类别编码神器): `s.map({'M': '男', 'F': '女'})`
- 也可传函数

DataFrame.apply

- `df.apply(func, axis=0)` :对每列(默认)
- `df.apply(func, axis=1)` :对每行

DataFrame.applymap / map

- 对每个元素应用(老版用 `applymap`,新版用 `map`)

transform

- 保持原 shape 的变换
- 经典用法:按组标准化
- `df.groupby('组')['列'].transform(lambda x: (x - x.mean()) / x.std())`

性能口诀

- 能向量化就别 `apply`
- `apply` 比向量化慢 100x+
- 大数据用 `swifter` / `numba` 加速

第 6 节:groupby ★★★★★

三步法

- `split`:按某列分组
- `apply`:对每组应用函数
- `combine`:合并结果

基础聚合

- `g['col'].mean/sum/count/std/var/median/min/max`
- `g['col'].first/last/nunique`
- `g.size()` :每组样本数(包括 NaN)
- `g['col'].quantile(0.25)`

多列分组

- `df.groupby(['列1', '列2'])`

.agg() 多统计量

- `g['col'].agg(['mean', 'sum', 'std'])` :列表
- `g['col'].agg(平均='mean', 总和='sum')` :命名聚合
- `g.agg({'A': 'mean', 'B': ['sum', 'std']})` :不同列不同函数

.transform() 广播回原 shape

- `g['col'].transform('mean')` :每行对应"组的均值"
- 经典:按组标准化、组内 demean、按组填缺失

.filter() 按组筛选

- `g.filter(lambda group: len(group) > 5)`

自定义聚合

- `g['col'].agg(lambda x: x.max() - x.min())`
- `g['col'].agg(极差=lambda x: x.max() - x.min())`

应统场景

- 按组标准化(z-score)
- 去组均值(固定效应)
- 按组填补缺失
- 滚动窗口的"按客户分组"

第 7 节:合并与重塑

merge:SQL JOIN 风格

- `pd.merge(L, R, on='键', how='...')`

- how: `inner` / `left` / `right` / `outer`
- 列名不同: `left_on=, right_on=`
- 用 index 作键: `left_index=True, right_index=True`
- 防止冲突: `suffixes=('_L', '_R')`
- 验证关系: `validate='many_to_one'`

concat: 简单拼接

- `pd.concat([df1, df2])`: 纵向(默认 axis=0)
- `pd.concat([df1, df2], axis=1)`: 横向
- `ignore_index=True`: 重置 index

pivot_table: 透视表

- `df.pivot_table(values=, index=, columns=, aggfunc=, margins=True)`
- 类似 Excel 透视表
- 多个 aggfunc 用列表
- margins 加行列总计

melt: 宽表转长表

- `df.melt(id_vars=['ID 列'], var_name='变量名', value_name='值')`
- 应用: 画图前置、面板回归数据准备

stack / unstack

- `df.stack()`: 列变成行的子级(变长表)
- `df.unstack()`: 反向操作

第 8 节: 时间序列 ★★★★★

转换

- `pd.to_datetime(s)`: 字符串 → 时间戳
- `pd.to_datetime(s, format='%Y/%m/%d', errors='coerce')`

dt 访问器

- `.dt.year/month/day/hour/minute`
- `.dt.dayofweek` (周一=0)
- `.dt.day_name()/month_name()`
- `.dt.quarter`
- `.dt.is_weekend`

DatetimeIndex

- `df.set_index('日期')`
- 字符串切片: `df['2024']` / `df['2024-03']` / `df['2024-03-15':'2024-03-31']`

resample 时间窗口聚合

- `df.resample('M').sum()` :按月
- `df.resample('W').mean()` :按周
- 频率别名:D/B/W/M/MS/Q/Y/H/T/S

rolling 滑动窗口

- `df['col'].rolling(window=7).mean()` :7 日移动平均
- `df['col'].rolling(window=30).std()` :30 日波动率
- `df['A'].rolling(60).corr(df['B'])` :滚动相关性
- `df['col'].expanding().sum()` :从开始的累计

shift 时间错位

- `df['col'].shift(1)` :滞后 1 期
- `df['col'].shift(-1)` :未来 1 期
- `df['col'].diff()` :一阶差分
- `df['col'].pct_change()` :增长率

生成日期范围

- `pd.date_range(start, end/periods=n, freq='D/M/Y')`

第 9 节:可视化 + EDA

Pandas 内置画图

- `df.plot(kind=...)`
- kind:line / bar / barh / hist / box / kde / area / scatter / pie
- subplots / figsize / title / grid

Seaborn(更专业)

- `sns.histplot/displot/kdeplot` :分布
- `sns.scatterplot/lmplot/regplot` :关系
- `sns.boxplot/violinplot/barplot/countplot` :分类
- `sns.heatmap(df.corr())` :热力图
- `sns.pairplot(df, hue='类别')` :全变量两两关系
- `sns.set_theme(style='whitegrid')` :全局美化

EDA 标准 7 步

1. 加载数据 + 看 shape/info/describe/head
2. 缺失值分析(数量、比例、模式)
3. 数值列分布(直方图、箱线图)
4. 类别列频次(value_counts)
5. 相关性分析(corr + heatmap)
6. 目标变量分析
7. 双变量分析(数值-数值散点、类别-数值小提琴)

10 大核心记忆点

1. **Pandas = 内存数据库 + 表格运算**,核心是 Series 和 DataFrame
2. **Index 是灵魂**:按标签做事,自动对齐
3. `.loc` (标签,右闭) vs `.iloc` (位置,右开)

4. 多条件用 `&` `|` `~` + 加括号
5. 缺失值优先填充,慎用 `dropna`
6. **groupby 是核心**:split-apply-combine,记 agg/transform/filter
7. 能向量化就别 `apply`
8. `merge` 等于 **SQL JOIN**,记四种 how
9. 设 **DatetimeIndex** 后开外挂:字符串切片 / `resample` / `rolling`
10. **EDA 流程要熟**:作为职业肌肉记忆

必踩坑

错	对
<code>df['A', 'B']</code>	<code>df[['A', 'B']]</code>
以为 <code>loc[0:5]</code> 不含 5	<code>loc</code> 右闭, <code>iloc</code> 才右开
<code>and</code> 写多条件	<code>&</code>
<code>df_subset['新列'] = 100</code> (警告)	<code>.copy()</code> 或 <code>.loc</code> 一次到位
用 <code>apply</code> 算简单运算	直接向量化
<code>to_csv</code> 没加 <code>index=False</code>	几乎总要 <code>index=False</code>
中文 CSV 报错	<code>encoding='gbk'/'gb18030'</code>
时间序列没设 <code>index</code>	必须 <code>set_index('日期')</code>
<code>groupby</code> 后想"广播回去"用了 <code>agg</code>	该用 <code>transform</code>
<code>merge</code> 不验证关系	<code>validate='many_to_one'</code>

应统人常用模式

```
# 按组 z-score
df.groupby('组')['col'].transform(lambda x: (x-x.mean())/x.std())

# 去组均值(固定效应)
df['col'] - df.groupby('组')['col'].transform('mean')

# 按组填缺失
df.groupby('组')['col'].transform(lambda x: x.fillna(x.mean()))

# 交叉频次表(类似 R 的 table)
pd.crosstab(df['A'], df['B'], normalize='index', margins=True)

# 相关性热力图
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', center=0)

# 滞后/滚动特征
df.groupby('客户')['销售'].shift(1)
df.groupby('客户')['销售'].transform(lambda x: x.rolling(30).mean())

# 透视看交叉
df.pivot_table(values=, index=, columns=, aggfunc=, margins=True)

# Pareto(累计占比)
df.sort_values('销售', ascending=False)['销售'].cumsum() / df['销售'].sum()
```

debug 心法

1. 出 bug 先 `print(df.shape, df.dtypes, df.head())`
2. 选择行有问题:搞清楚是 loc 还是 iloc
3. 链式赋值警告:用 `.copy()` 或 `.loc`
4. groupby 结果不对:多打印中间步骤
5. merge 后行数变多/变少:检查关系是否 1:1
6. 时间序列没生效:确认 index 是 DatetimeIndex